OS LAB 3

Write a C program to simulate ***SJF*** scheduling algorithm ***non preemptive***.

```c
#include<stdio.h>
#include<stdlib.h>
struct process
{
    int pid;
    int at;
    int bt;
    int ct;
    int tat;
    int wt;
    int done;
};
void sjf_np(struct process p[], int n)
{
    int completed, time;
    completed=0;
    time=0;
    while(completed<n)
    {
        int min_bt=9999;
        int index=-1;
        for(int i=0; i<n; i++)
        {
            if(p[i].at<=time && p[i].done==0 && p[i].bt<min_bt)
            {
                min_bt=p[i].bt;
                index=i;
            }
        }
        if(index==-1)
            time++;
        else
        {
            time+=p[index].bt;
            p[index].ct=time;
            p[index].tat=p[index].ct-p[index].at;
            p[index].wt=p[index].tat-p[index].bt;
            p[index].done=1;
            completed++;
        }
```

```
    }
}
void calcAvg(struct process p[], int n)
{
    float totaltat=0, totalwt=0;
    for(int i=0;i<n;i++)
    {
        totaltat+=p[i].tat;
        totalwt+=p[i].wt;
    }
    printf("\nAvg TAT: %f\nAvg WT: %f",(totaltat/n),(totalwt/n));
}
void display(struct process p[] , int n)
{
    if(!n)
    {
        printf("\nNo processes");
        return;
    }
    printf("\nPiD\tAT\tBT\tCT\tTAT\tWT");
    for(int i=0; i<n; i++)
    {
        printf("\n%d\t%d\t%d\t%d\t%d\t%d",p[i].pid,p[i].at,p[i].bt,p[i].ct,p[i].t
at,p[i].wt);
    }
    calcAvg(p,n);
}
int main()
{
    int n;
    printf("\nEnter the number of processes: ");
    scanf("%d",&n);
    struct process p[n];
    for(int i=0; i<n; i++)
    {
        p[i].pid=i+1;
        printf("\nEnter AT and BT for P%d: ",p[i].pid);
        scanf("%d%d",&p[i].at,&p[i].bt);
        p[i].done=0;
    }
    sjf_np(p,n);
    display(p,n);
}
```

OUTPUT:

```
Enter the number of processes: 5

Enter AT and BT for P1: 0 3

Enter AT and BT for P2: 2 2

Enter AT and BT for P3: 3 5

Enter AT and BT for P4: 4 4

Enter AT and BT for P5: 6 1

PiD     AT      BT      CT      TAT     WT
1       0       3       3       3       0
2       2       2       5       3       1
3       3       5       15      12      7
4       4       4       9       5       1
5       6       1       10      4       3
Avg TAT: 5.400000
Avg WT: 2.400000
```

Write a C program to simulate ***SJF*** scheduling algorithm ***preemptive***.

```c
#include<stdio.h>
#include<stdlib.h>
struct process
{
    int pid;
    int at;
    int bt;
    int ct;
    int tat;
    int wt;
    int rem;
    int done;
};
void sjf_pre(struct process p[], int n)
{
    int completed, time;
    completed=0;
    time=0;
    while(completed<n)
    {
```

```c
        int min_rem=9999;
        int index=-1;
        for(int i=0; i<n; i++)
        {
            if(p[i].at<=time && p[i].done==0 && p[i].rem<min_rem)
            {
                min_rem=p[i].rem;
                index=i;
            }
        }
        if(index==-1)
            time++;
        else
        {
            p[index].rem--;
            time++;
            if(p[index].rem == 0)
            {
                p[index].ct=time;
                p[index].tat=p[index].ct-p[index].at;
                p[index].wt=p[index].tat-p[index].bt;
                p[index].done=1;
                completed++;
            }
        }
    }
}
void calcAvg(struct process p[], int n)
{
    float totaltat=0, totalwt=0;
    for(int i=0;i<n;i++)
    {
        totaltat+=p[i].tat;
        totalwt+=p[i].wt;
    }
    printf("\nAvg TAT: %f\nAvg WT: %f",(totaltat/n),(totalwt/n));
}
void display(struct process p[] , int n)
{
    if(!n)
    {
        printf("\nNo processes");
        return;
    }
    printf("\nPiD\tAT\tBT\tCT\tTAT\tWT");
```

```
    for(int i=0; i<n; i++)
    {
        printf("\n%d\t%d\t%d\t%d\t%d\t%d",p[i].pid,p[i].at,p[i].bt,p[i].ct,p[i].t
at,p[i].wt);
    }
    calcAvg(p,n);
}
int main()
{
    int n;
    printf("\nEnter the number of processes: ");
    scanf("%d",&n);
    struct process p[n];
    for(int i=0; i<n; i++)
    {
        p[i].pid=i+1;
        printf("\nEnter AT and BT for P%d: ",p[i].pid);
        scanf("%d%d",&p[i].at,&p[i].bt);
        p[i].rem=p[i].bt;
        p[i].done=0;
    }
    sjf_pre(p,n);
    display(p,n);
}
```

OUTPUT:

```
Enter the number of processes: 5

Enter AT and BT for P1: 0 3

Enter AT and BT for P2: 2 2

Enter AT and BT for P3: 3 5

Enter AT and BT for P4: 4 4

Enter AT and BT for P5: 6 1

PiD     AT      BT      CT      TAT     WT
1       0       3       3       3       0
2       2       2       5       3       1
3       3       5       15      12      7
4       4       4       10      6       2
5       6       1       7       1       0
Avg TAT: 5.000000
Avg WT: 2.000000
```

Write a C program to simulate ***priority*** scheduling algorithm ***non preemptive***.

```c
#include<stdio.h>
#include<stdlib.h>
struct process
{
    int pid;
    int at;
    int bt;
    int priority;
    int ct;
    int tat;
    int wt;
    int done;
};
void priority_np(struct process p[], int n)
{
    int completed, time;
    completed=0;
    time=0;
    while(completed<n)
    {
        int min_priority=9999;
        int index=-1;
        for(int i=0; i<n; i++)
        {
            if(p[i].at<=time && p[i].done==0)
            {
                if(index == -1 || p[i].priority<min_priority ||
(p[i].priority==min_priority && p[i].at<p[index].at))
                {
                    min_priority=p[i].priority;
                    index=i;
                }
            }
        }
        if(index==-1)
            time++;
        else
        {
            time+=p[index].bt;
            p[index].ct=time;
            p[index].tat=p[index].ct-p[index].at;
            p[index].wt=p[index].tat-p[index].bt;
            p[index].done=1;
```

```
            completed++;
        }
    }
}
void calcAvg(struct process p[], int n)
{
    float totaltat=0, totalwt=0;
    for(int i=0;i<n;i++)
    {
        totaltat+=p[i].tat;
        totalwt+=p[i].wt;
    }
    printf("\nAvg TAT: %f\nAvg WT: %f",(totaltat/n),(totalwt/n));
}
void display(struct process p[] , int n)
{
    if(!n)
    {
        printf("\nNo processes");
        return;
    }
    printf("\nPiD\tAT\tBT\tP\tCT\tTAT\tWT");
    for(int i=0; i<n; i++)
    {
        printf("\n%d\t%d\t%d\t%d\t%d\t%d\t%d",p[i].pid,p[i].at,p[i].bt,p[i].prior
ity,p[i].ct,p[i].tat,p[i].wt);
    }
    calcAvg(p,n);
}
int main()
{
    int n;
    printf("\nEnter the number of processes: ");
    scanf("%d",&n);
    struct process p[n];
    for(int i=0; i<n; i++)
    {
        p[i].pid=i+1;
        printf("\nEnter AT, BT and priority for P%d: ",p[i].pid);
        scanf("%d%d%d",&p[i].at,&p[i].bt,&p[i].priority);
        p[i].done=0;
    }
    priority_np(p,n);
    display(p,n);
}
```

OUTPUT:

```
Enter the number of processes: 5

Enter AT, BT and priority for P1: 0 3 5

Enter AT, BT and priority for P2: 2 2 4

Enter AT, BT and priority for P3: 3 5 1

Enter AT, BT and priority for P4: 4 4 2

Enter AT, BT and priority for P5: 6 1 0

PiD     AT      BT      P       CT      TAT     WT
1       0       3       5       3       3       0
2       2       2       4       15      13      11
3       3       5       1       8       5       0
4       4       4       2       13      9       5
5       6       1       0       9       3       2
Avg TAT: 6.600000
Avg WT: 3.600000
```

Write a C program to simulate ***priority*** scheduling algorithm ***preemptive***.

```c
#include<stdio.h>
#include<stdlib.h>
struct process
{
    int pid;
    int at;
    int bt;
    int priority;
    int ct;
    int tat;
    int wt;
    int rem;
    int done;
};
void priority_pre(struct process p[], int n)
{
    int completed, time;
    completed=0;
    time=0;
    while(completed<n)
    {
        int min_priority=9999;
```

```
        int index=-1;
        for(int i=0; i<n; i++)
        {
            if(p[i].at<=time && p[i].done==0)
            {
                if(index == -1 || p[i].priority<min_priority ||
(p[i].priority==min_priority && p[i].at<p[index].at))
                {
                    min_priority=p[i].priority;
                    index=i;
                }
            }
        }
        if(index==-1)
            time++;
        else
        {
            p[index].rem--;
            time++;
            if(p[index].rem==0)
            {
                p[index].ct=time;
                p[index].tat=p[index].ct-p[index].at;
                p[index].wt=p[index].tat-p[index].bt;
                p[index].done=1;
                completed++;
            }
        }
    }
}
void calcAvg(struct process p[], int n)
{
    float totaltat=0, totalwt=0;
    for(int i=0;i<n;i++)
    {
        totaltat+=p[i].tat;
        totalwt+=p[i].wt;
    }
    printf("\nAvg TAT: %f\nAvg WT: %f",(totaltat/n),(totalwt/n));
}
void display(struct process p[] , int n)
{
    if(!n)
    {
        printf("\nNo processes");
```

```
        return;
    }
    printf("\nPiD\tAT\tBT\tP\tCT\tTAT\tWT");
    for(int i=0; i<n; i++)
    {
        printf("\n%d\t%d\t%d\t%d\t%d\t%d\t%d",p[i].pid,p[i].at,p[i].bt,p[i].prior
ity,p[i].ct,p[i].tat,p[i].wt);
    }
    calcAvg(p,n);
}
int main()
{
    int n;
    printf("\nEnter the number of processes: ");
    scanf("%d",&n);
    struct process p[n];
    for(int i=0; i<n; i++)
    {
        p[i].pid=i+1;
        printf("\nEnter AT, BT and priority for P%d: ",p[i].pid);
        scanf("%d%d%d",&p[i].at,&p[i].bt,&p[i].priority);
        p[i].rem=p[i].bt;
        p[i].done=0;
    }
    priority_pre(p,n);
    display(p,n);
}
```

OUTPUT:

```
Enter the number of processes: 5

Enter AT, BT and priority for P1: 0 3 5

Enter AT, BT and priority for P2: 2 2 4

Enter AT, BT and priority for P3: 3 5 1

Enter AT, BT and priority for P4: 4 4 2

Enter AT, BT and priority for P5: 6 1 0

PiD     AT      BT      P       CT      TAT     WT
1       0       3       5       15      15      12
2       2       2       4       14      12      10
3       3       5       1       9       6       1
4       4       4       2       13      9       5
5       6       1       0       7       1       0
Avg TAT: 8.600000
Avg WT: 5.600000
```